

College of Computing

Computer Science Department

Project Event Management CSC382

- Student Information

Student 1:	**Salem Hamdan Salem Al-Omrani**	**ID:**	**202322090**
Student 2:	**Abdullah Ali Khalaf Al-Rashidi**	**ID:**	**202422065**
Student 3:	**Ayman Abdullah Al-Omari**	**ID:**	**202412014**

Project Management Plan for the Event Management System (EMS)

1. Introduction

This Project Management Plan (PMP) outlines the structured approach for the successful execution and delivery of the Event Management System (EMS) project. The plan encompasses three critical areas: detailed project planning, comprehensive risk management, and clear definition of team roles and responsibilities [1]. The methodology employed is aligned with industry best practices, ensuring that the project is delivered on time, within scope, and to the required quality standards, as defined in the Software Requirements Specification (SRS) [2].

2. Project Planning

Effective project planning is essential for translating the project scope into actionable tasks and a measurable timeline. This section details the Work Breakdown Structure (WBS) and the corresponding project schedule.

2.1. Work Breakdown Structure (WBS)

The WBS is a hierarchical decomposition of the total scope of work to be carried out by the project team to accomplish the project objectives and create the required deliverables [3]. For the EMS, the WBS is structured into four main phases, reflecting a typical small-scale web application development lifecycle.

ID	WBS Element	Description	Deliverable
1.0	**Initiation & Planning**	Define project goals, scope, and initial requirements.	SRS Document, Project Plan (PMP)
1.1	Requirements Elicitation & Analysis	Finalize functional and non-functional requirements.	Finalized Requirements List
1.2	Database Design	Design the MySQL database schema (tables, relationships).	event_management.sql script
1.3	UI/UX Design	Create wireframes and final design for key pages (e.g., event view, admin dashboard).	styles.css and UI Mockups
2.0	**Development & Implementation**	Coding and integration of all system components.	Fully Coded System
2.1	Backend Development (PHP)	Implement core logic: authentication (auth.php), database connection (db_connect.php), and helper functions (helpers.php).	Core PHP Modules
2.2	Frontend Development (UI)	Implement user interfaces for all pages (index.php, login.php, pages/*).	HTML/PHP Templates
2.3	CRUD Functionality	Implement Create, Read, Update, Delete (CRUD) operations for events and registration.	Event Management Modules
3.0	**Testing & Quality Assurance (QA)**	Verify that the system meets all specified requirements.	Test Reports, Defect Log
3.1	Unit Testing	Test individual functions and modules (e.g., authentication, database queries).	Unit Test Results
3.2	Integration Testing	Test the interaction between different modules (e.g., registration form to database).	Integration Test Results

3.3	Security Testing	Verify security measures (CSRF, XSS, SQL Injection prevention).	Security Audit Report
4.0	**Deployment & Closure**	Final system setup and project handover.	Deployed System, Final Report
4.1	Production Deployment	Configure the web server and deploy the final code and database.	Live System
4.2	Documentation & Training	Finalize user and technical documentation.	User Manual, Technical Documentation

2.2. Project Schedule (Gantt Chart)

The project schedule, visualized as a Gantt Chart, provides a timeline for the execution of the WBS elements. This schedule assumes a total project duration of 8 weeks, with tasks running sequentially and in parallel where possible [4].

[Arabic comment: Insert a Gantt Chart diagram showing the WBS elements (1.0 to 4.2) on the Y-axis and the 8-week timeline on the X-axis. Highlight the critical path, which includes Database Design, Backend Development, CRUD Functionality, and Security Testing.]

WBS ID	Task Name	Duration (Weeks)	Start Week	End Week	Dependencies
1.0	Initiation & Planning	2	1	2	None
1.2	Database Design	1	2	2	1.1
1.3	UI/UX Design	2	1	2	1.1
2.1	Backend Development	2	3	4	1.2
2.2	Frontend Development	2	3	4	1.3
2.3	CRUD Functionality	2	5	6	2.1, 2.2
3.1	Unit Testing	1	6	6	2.3
3.3	Security Testing	1	7	7	2.3
4.1	Production Deployment	1	8	8	3.3

3. Risk Management

Risk management is the process of identifying, assessing, and controlling threats to a project's capital and earnings [5]. For the EMS project, a proactive approach is adopted, focusing on identification and the development of mitigation strategies.

3.1. Risk Identification and Assessment

The following table identifies the key potential risks, assesses their impact and probability, and calculates the risk exposure (Impact x Probability) [6].

ID	Risk Description	Probability (P)	Impact (I)	Exposure (P x I)	Category
R-1	**Scope Creep:** Uncontrolled changes or additions to the project scope after the SRS is finalized.	Medium (0.5)	High (0.8)	0.40	Planning
R-2	**Security Vulnerability:** A critical security flaw (e.g., XSS, SQLi) is discovered post-deployment.	Medium (0.6)	High (0.9)	0.54	Technical
R-3	**Technical Debt:** Poorly written or undocumented code requires significant rework.	Low (0.3)	Medium (0.7)	0.21	Technical
R-4	**Resource Unavailability:** Key team member is unavailable during a critical development phase.	Low (0.2)	High (0.8)	0.16	External
R-5	**Database Performance:** Slow query execution due to poor database design or high load.	Medium (0.4)	Medium (0.6)	0.24	Technical

3.2. Risk Mitigation Plan

A mitigation plan is developed for the highest-exposure risks (R-1 and R-2) to reduce their probability or impact [7].

Risk ID	Mitigation Strategy	Risk Response Type

R-1 (Scope Creep)	**Avoidance/Reduction:** Implement a formal change control process where all scope changes must be documented, assessed for impact, and approved by the Project Manager and Stakeholder before implementation.	Avoidance
R-2 (Security Vulnerability)	**Reduction:** Integrate mandatory security code reviews and dedicated Security Testing (WBS 3.3) into the schedule. Enforce the use of Prepared Statements and CSRF tokens as non-functional requirements.	Reduction
R-3 (Technical Debt)	**Reduction:** Enforce coding standards and mandatory peer code reviews (WBS 3.1) to ensure code quality and maintainability.	Reduction
R-5 (Database Performance)	**Reduction:** Conduct performance testing on all critical database queries and enforce indexing on frequently searched columns (e.g., event title, date) during the Database Design phase (WBS 1.2).	Reduction

4. Team Roles and Responsibilities

For a small-scale web development project like the EMS, a lean team structure is optimal, where individuals often take on multiple roles [8]. Clear definition of responsibilities ensures accountability and efficient workflow [9].

Role	Assigned To (Student Name)	Student ID	Key Responsibilities
Project Manager (PM)	Ayman Abdullah Al-Amri	202412014	Overall project oversight, stakeholder communication, risk management (R-1), and schedule tracking.
Lead Developer (Full-Stack)	Salem Hamdan Salem Al-Omrani	202322090	Primary responsibility for all coding (PHP, MySQL, HTML/CSS/JS), database design (WBS 1.2), and implementation of core functionality (WBS 2.1, 2.3).
Quality Assurance (QA) Engineer	Abdullah Ali Khalaf Al-Rashidi	202422065	Design and execute all testing activities (WBS 3.1, 3.2, 3.3), including security testing (R-2), and manage the defect log.

Technical Writer/Analyst	Ayman Abdullah Al-Amri	202412014	Responsible for creating and maintaining all project documentation, including the SRS, PMP, and final user manuals (WBS 4.2).

The Lead Developer and QA Engineer roles are critical for the successful execution of the technical WBS elements, while the Project Manager maintains the strategic alignment and manages external risks [10].

References

[1]: Project Management Institute (PMI). (2021). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)* – Seventh Edition. [2]: IEEE. (1998). *IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)*. [3]: PMI. (n.d.). *Work Breakdown Structure (WBS) - Basic Principles*. [https://www.pmi.org/learning/library/work-breakdown-structure-basic-principles-4883]. [4]: GanttPRO. (n.d.). *Work Breakdown Structure Examples (WBS) for Various Projects*. [https://blog.ganttpro.com/en/work-breakdown-structure-example-wbs/]. [5]: Atlassian. (n.d.). *What is Risk Mitigation? Project Management Strategies*. [https://www.atlassian.com/work-management/project-management/risk-mitigation]. [6]: Netguru. (2024). *Effective Strategies for Software Risk Management*. [https://www.netguru.com/blog/strategies-for-software-risk-management]. [7]: Saviom. (2024). *Top 12 IT Project Risks: Effective Ways to Mitigate Them*. [https://www.saviom.com/blog/10-common-it-project-risks-ways-to-mitigate-them/]. [8]: Alcor BPO. (2025). *11 Key Roles in a Software Development Team*. [https://alcor-bpo.com/10-key-roles-in-a-software-development-team-who-is-responsible-for-what/]. [9]: Deazy. (n.d.). *Software Development Team Structure - Important Roles*. [https://www.deazy.com/knowledge-hub/software-development-team-roles-and-responsibilities]. [10]: HireFraction. (2024). *Optimal Dev Team Structure for Small Teams*. [https://www.hirefraction.com/blog/optimal-dev-team-structure-for-small-teams]. [11]: Project-Management.com. (2022). *Work Breakdown Structure (WBS) for IT Projects*. [https://project-management.com/work-breakdown-structure-for-it-projects-wbs/]. [12]: Energy.gov. (2000). *Software Risk Management A Practical Guide*. [https://energy.gov/sites/prod/files/cioprod/documents/Risk_Management.pdf].